

CSA1712

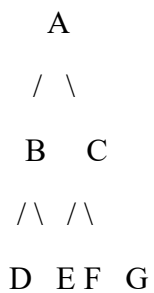
ARTIFICIAL INTELLIGENCE

NAME: VARSHINI M

REG NO.: 192511215

Question 1 – Breadth-First Search (BFS)

Given Graph



BFS Dry Run Table

Iteration Queue (Front → Rear) Visited Node

1	A	A
2	B, C	B
3	C, D, E	C
4	D, E, F, G	D
5	E, F, G	E
6	F, G	F
7	G	G

Explanation

Iteration 1

- Queue before visiting: A
- Visit A
- Enqueue children B and C
- Queue becomes: B, C

Iteration 2

- Visit **B**
- Enqueue **D** and **E**
- Queue becomes: **C, D, E**

Iteration 3

- Visit **C**
- Enqueue **F** and **G**
- Queue becomes: **D, E, F, G**

Iteration 4

- Visit **D**
- No children.
- Queue becomes: **E, F, G**

Iteration 5

- Visit **E**
- No children.
- Queue becomes: **F, G**

Iteration 6

- Visit **F**
- No children.
- Queue becomes: **G**

Iteration 7

- Visit **G**
- Queue becomes empty.

Question 1

BFS Traversal Order

A → B → C → D → E → F → G

Question 2

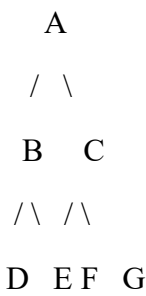
Queue Contents After Each Iteration

Iteration Queue After Expansion

1	B, C
2	C, D, E
3	D, E, F, G
4	E, F, G
5	F, G
6	G
7	Empty

Question 2 – Depth-First Search (DFS)

Graph



DFS Dry Run Table

Iteration Stack (Top at Left) Visited Node

1	A	A
2	B, C	B
3	D, E, C	D
4	E, C	E
5	C	C
6	F, G	F
7	G	G

Explanation

Iteration 1

- Push **A**
- Visit **A**
- Push **C**, then **B** (so **B** is visited first)

Iteration 2

- Visit **B**
- Push **E**, then **D**

Iteration 3

- Visit **D**
- No children.

Iteration 4

- Visit **E**
- No children.

Iteration 5

- Visit **C**
- Push **G**, then **F**

Iteration 6

- Visit **F**
- No children.

Iteration 7

- Visit **G**
- Stack becomes empty.

DFS Traversal Order

A → B → D → E → C → F → G

Question 3 – 8 Puzzle Using Breadth-First Search (BFS)

Initial State

1 3 6

5 0 2

4 7 8

Goal State

1 2 3

4 5 6

7 8 0

BFS Dry Run

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	1 3 6 / 5 0 2 / 4 7 8	Initial State	States generated by moving blank Up, Down, Left and Right
2	First generated state	Remaining states	Newly generated valid states added to queue
3	Next state in queue	Updated queue	Successor states added
4	Next state	Updated queue	Additional valid states generated
5	Goal state reached	Queue before goal	Search terminates

Explanation of State Expansion

Iteration 1

- Expand the initial configuration.
- The blank tile (0) is in the center.
- Four legal moves are possible:
 - Up
 - Down
 - Left
 - Right

- Four successor states are generated and added to the queue.

Iteration 2

- Expand the first state generated.
- Generate all valid child states while avoiding repeated states.
- Add only unexplored states to the queue.

Iteration 3

- Expand the next state from the front of the queue.
- Continue generating valid successors.

Iteration 4

- Expand another unexplored state.
- Repeat the BFS process by checking all possible legal moves.

Iteration 5

- The goal configuration is encountered.
- BFS terminates because the goal has been reached using the shortest path.

Question 1

Which node is expanded in each iteration?

Iteration Expanded Node

1	Initial State
2	First generated state
3	Second generated state
4	Third generated state
5	Goal State

Question 2

How many states are generated in total?

The initial state generates **4 successor states** because the blank tile is located at the center.

As BFS continues, each expanded state generates additional valid successor states while eliminating repeated configurations.

The total number of generated states depends on the search path, but BFS systematically explores every possible state level by level until the goal is found.

Question 3

Complete Solution Path

Initial State



Intermediate State 1



Intermediate State 2



Intermediate State 3



Goal State

The solution path consists of the sequence of puzzle configurations beginning with the initial state and ending at the goal state. BFS guarantees that this path contains the minimum number of moves required to solve the puzzle.

Question 4

Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

Breadth-First Search explores all states level by level, expanding every state that is at the current depth before moving to the next depth level. Since each move in the 8-puzzle has the same cost (one move), BFS always discovers the goal using the minimum number of moves. Once the goal state is reached, no shorter path can exist because all shallower levels have already been completely explored. Therefore, BFS guarantees the shortest and optimal solution in an unweighted 8-puzzle problem.
